

Functional Programming

Radu Răzvan Slăvescu

Technical University of Cluj-Napoca
Department of Computer Science

Outline

Monads

Input and output

Parallelism via Eval Monad

Monads in a nutshell

What is a Monad?

Monad = an `Applicative` that supports two more operations:

```
class Applicative m => Monad m where
  (>>=)  :: m a -> (a -> m b) -> m b    --"bind"

  return :: a -> m a -- for backwards compatib.
  return = pure
```

Remark

A Monad knows how to apply a function which takes a normal value and returns a wrapped value to a wrapped value.

Monads in a nutshell

Making `Maybe` into a `Monad`

```
data Maybe a = Nothing | Just a
```

```
instance Monad Maybe where
```

```
  (Just x) >>= k = k x
```

```
  Nothing  >>= _ = Nothing
```

Exception Handling with Maybe

A Monadic function

```
mbHalf :: Int -> Maybe Int
mbHalf a | even a      = Just (div a 2)
         | otherwise = Nothing
```

Example

```
> Just 22 >= mbHalf
Just 11
> Just 22 >= mbHalf >= mbHalf
Nothing
> Just 22 >= mbHalf >= mbHalf >= mbHalf
Nothing
```

Exception Handling with Maybe

A Monadic function

```
mbHalf :: Int -> Maybe Int
mbHalf a | even a      = Just (div a 2)
         | otherwise = Nothing
```

```
> Just 22 >>= mbHalf >>= mbHalf >>= mbHalf
Nothing
```

Remark: the `error`-based mechanism is too drastic!

The `Maybe` monad allows handling exceptions more gracefully: operator `(>>=)` allows chaining operations that might fail, but the code is written as if it follows the "happy path". Errors, if any, are sent downstream to be handled at the end of the chain.

Monads in a nutshell

Making List into a Monad

```
instance Monad [] where
-- (>>=) :: [a] -> (a -> [b]) -> [b]
xs >>= g = [y | x <- xs, y <- g x]
-- maps g, then flattens result
```

Example

```
> ["ann", "has", "figs"] >>= (\ w -> [length w])
[3,3,4]
> :m Data.Char
> ["Java", "style", "ID"] >>=
    (\ (x:xs) -> (toUpper x):(toLower <$> xs))
"JavaStyleId"
```

Container vs. IO type

Does functor f have access to the structure of $f\ a$?

1. yes: e.g., if $f\ a$ is a *container* (e.g., `List`, `Maybe` etc.)
2. no: e.g., if $f\ a$ represents input/output actions

Container vs. IO type

Question

How to deal with interactive programs (i.e., I/O side effects)?

Definition (An interactive program in Haskell)

It is a **pure** function which maps the *current* state of the world to a *modified* state of the world (where the latter includes all side effects of the program) and also returns a value of type a :

$$\text{type IO } a = \text{World} \rightarrow (a, \text{World})$$

Remark

IO signals "interaction with the outside world".

Container vs. IO type

Definition (Actions)

They are expressions of type `IO a`.

Example (Type `IO a` actions)

- ▶ `IO Char`: actions returning a character
- ▶ `IO ()`: purely side-effected; returns no meaningful value (still, a dummy value gets returned: the nullary tuple `()`, aka unit value, which is the only value in the type `Unit`)

Example (Read a `Char` and return an `Int`)

Type: `Char -> IO Int`

This is a shortcut for: `Char -> World -> (Int, World)`

Basic actions

Example (getChar)

`getChar :: IO Char`

Reads a character, echoes it and returns it wrapped in `IO`

Example (putChar c)

`putChar :: Char -> IO ()`

Writes the character `c` on the screen and returns `IO ()`

Example (return v)

`return :: a -> IO a`

Returns value `v`, wrapped in `IO`, with no interaction w/ the user (it provides a bridge from *pure* expressions to *impure* actions, i.e. from no side effects to side effects)

Basic actions

Example (`putStrLn s`)

`putStrLn :: String -> IO ()`

Prints the string `s` on the screen and returns `IO ()`

Example (`getLine :: IO String`)

Reads a string from the outside world and returns it wrapped in `IO`, so one cannot simply access it. To access the string, one can use operator `(>>=)`, which can extract it from its "pack".

The `do` notation

Combining a sequence of `IO` actions into a composite one

```
do  v1 <- a1
    v2 <- a2  -- generator
    ...
    vn <- an
    return (g v1 v2 ... vn)
```

Perform `a1` to get the result `v1`, ..., then `an` to get `vn`, then apply `g` and combine all results into one value to be returned

Remarks

- ▶ expression `vi <- ai` is called "generator"
- ▶ `_ <- ai` is abbreviated `ai` (if `vi` not actually needed)

The `do` notation

Example (`getLine`: read a string from keyboard)

```
getLine :: IO String
getLine = do x <- getChar
            if x == '\n' then
                return []
            else
                do xs <- getLine
                   return (x:xs)
```

Example (`putStr`: write a string to the screen)

```
putStr :: String -> IO ()
putStr []      = return ()
putStr (x:xs) = do putChar x
                   putStr xs
```

Dealing with IO

Making IO into a Functor

```
instance Functor IO where
  --fmap :: (a -> b) -> IO a -> IO b
  fmap g mx = do  x <- mx
                  return (g x)
```

Remark

`fmap` applies `g` to the result value of the argument action.

Example (`fmap` for IO)

```
> fmap show (return True)
"True"
```

Dealing with IO

Making IO into an Applicative

```
instance Applicative IO where
  --pure :: a -> IO a
  pure = return

  -- (<*>) :: IO (a -> b) -> IO a -> IO b
  mg <*> mx = do g <- mg
                 x <- mx
                 return (g x)
```

Remark

The `<*>` operator for the `IO` applicative applies an impure function to an impure argument to get an impure result

Dealing with IO

Example (Read n characters from the keyboard)

```
getChrs :: Int -> IO String
getChrs 0 = return []
getChrs n = pure (:) <*> getChar <*> getChrs (n-1)
```

Remark

Applicative style for IO supports *interactive* programming: pure functions can be applied to impure arguments. Action sequencing and result value extraction are done automatically

Dealing with IO

Three things on `Applicatives`

1. generalize mapping to functions with multiple arguments
2. apply pure functions to effectful arguments (which may fail, perform input/output actions etc)
3. allow defining generic functions, which can be used with any applicative functor

Dealing with IO

Definition (sequenceA)

```
sequenceA :: Applicative f => [f a] -> f [a]
sequenceA []          = pure []
sequenceA (x:xs) = pure (:) <*> x <*> sequenceA xs
```

Example (getChrs, simpler)

```
getChrs :: Int -> IO String
getChrs n = sequenceA (replicate n getChar)
```

Interacting with IO

Example (Displaying strings)

```
main :: IO ()  
main = putStrLn " Good job!"
```

```
> main  
Good Job!
```

Interacting with IO

Example (Read a string, then display it)

```
main :: IO ()
main = getLine >>=
      \ n -> putStrLn ("Read " ++ n ++ " done.")
```

```
> main
4 -- <-- entered by user
Read 4 done.
```

Interacting with IO

Example (Display message, read a string, then display it)

```
main :: IO ()
main =  putStrLn "Enter number" >>=
        (\ _ -> getLine >>=
          (\ s -> putStrLn ("Read " ++ s ++ " done."

> main
Enter number
4 -- <-- entered by user
Read 4 done.
```

Interacting with IO

Example (Display message, read a string, then display it)

```
main :: IO ()
main = do
    putStrLn "Enter number"
    s <- getLine
    putStrLn ("Read " ++ s ++ " done.")
```

```
> main
Enter number
4 -- <-- entered by user
Read 4 done.
```

Interacting with IO

Example (Read 2 strings, then display them)

```
main :: IO ()
main = do
    putStrLn "Enter numerator"
    sn <- getLine
    putStrLn "Enter denominator"
    sd  <- getLine
    putStrLn (sn ++ "/" ++ sd ++ " done.")
```

```
> main
Enter numerator
8
Enter denominator
12
8/12 done.
```

Interacting with IO

Example (Read 2 strings, then display them)

```
main :: IO ()
main = putStrLn "Numerator:" >>=
      (\ _ -> getLine >>=
        (\ sn -> putStrLn "Denominator:" >>=
          (\ _ -> getLine >>=
            (\ sd ->
              putStrLn (sn ++ "/" ++ sd ++ " done."))
```

```
> main
```

```
Enter numerator
```

```
8
```

```
Enter denominator
```

```
12
```

```
8/12 done.
```

Interacting with IO

Example (Extract integers)

```
getNumber :: IO Integer
getNumber = readLn
```

```
main :: IO ()
main = do
    putStrLn "Numerator"
    n <- getNumber
    putStrLn "Denominator"
    d <- getNumber
    let g = gcd n d
    let n1 = n `div` g
    let d1 = d `div` g
    putStrLn ((show n1) ++ "/" ++
               (show d1) ++ " done.")
```

... same input as before ... 2/3 done.

Monads and arithmetic expressions

Example (Implementing division)

```
data Expr = Val Int | Div Expr Expr
```

```
eval :: Expr -> Int
```

```
eval (Val n)    = n
```

```
eval (Div x y) = eval x `div` eval y
```

Remark

If `y` is 0, a divide by zero exception is raised. What to do?

Monads and arithmetic expressions

Example (Implementing division, take 2: use `Maybe`)

```
safediv :: Int -> Int -> Maybe Int
safediv _ 0 = Nothing
safediv n m = Just (n `div` m)
```

```
eval :: Expr -> Maybe Int
eval (Val n)    = Just n
eval (Div x y) = case (eval x) of
    Nothing -> Nothing
    Just n   -> case (eval y) of
        Nothing -> Nothing
        Just m   -> safediv n m
```

Remark

This is verbose. How to simplify?

Monads and arithmetic expressions

Example (Implementing division. Simpler?!)

```
eval :: Expr -> Maybe Int
eval (Val n)      = pure n
eval (Div x y)    = pure safediv <*> eval x <*> eval y
```

Remark

Not type correct: `safediv` returns `Maybe Int`, not `Int`!

We need something which has an argument of a type `a` that might fail and a function of type `a -> b` **which might fail** and returns a result of type `b` which might fail. But this is **a monad**!

Monads and arithmetic expressions

Example (Implementing division, take 3: `employ >>=`)

```
eval :: Expr -> Maybe Int
eval (Val n)      = Just n
eval (Div x y)    = eval x >>=
                    \ n -> eval y >>=
                    \ d -> safediv n d
```

Remark

`>>=` abstracts out the case analysis which is done twice before.
It combines sequencing of values with processing their results

Monads and arithmetic expressions

Example (Implementing division, take 4: `do` notation)

```
eval :: Expr -> Maybe Int
eval (Val n)    = Just n
eval (Div x y)  = eval x >>=
                  \ n -> eval y >>=
                  \ d -> safediv n d

eval (Val n)    = Just n
eval (Div x y)  = do n <- eval x
                  d <- eval y
                  safediv n d
```

Remark

The `do` notation, with the precise meaning illustrated above provides syntax sugar and can be used with any monad. Because of `>>=`, `do` succeeds iff each component succeeds.

Generic Functions

Advantage of abstracting out the `Monad` concept

Write generic functions, i.e. can be used with any monad

Example (Monadic `map`)

```
mapM :: Monad m => (a -> m b) -> [a] -> m [b]
mapM g [] = return []
mapM g (x:xs) = do y <- g x
                  ys <- mapM g xs
                  return (y:ys)
```

Remark

Like an ordinary `map`, but both the argument function and `mapM` return monadic types.

Generic Functions

Example (Monadic `map`)

```
mapM :: Monad m => (a -> m b) -> [a] -> m [b]
mapM g [] = return []
mapM g (x:xs) = do y <- g x
                   ys <- mapM g xs
                   return (y:ys)
```

```
print :: Show a => a -> IO ()
print = putStrLn . show
```

```
> mapM print [1,2,3]
1
2
3
[(), (), ()] -- returned value
```

Generic Functions

Example (Don't care about the returned result: `mapM_`)

```
> mapM_ print [1,2,3]
```

```
1
```

```
2
```

```
3
```

Generic Functions

Example (Monadic filter)

```
filtM :: Monad m => (a -> m Bool) -> [a] -> m [a]
filtM p [] = return []
filtM p (x:xs) = do b <- p x
                    ys <- filtM p xs
                    return (if b then x:ys else ys)
```

```
> filtM (\ b -> [False, True]) [3,2,1]
1p if you explain WHY
```

Need for speed

- ▶ Moore law and multi-core
- ▶ `Eval` monad in Haskell
- ▶ source: Simon Marlow, "Parallel and Concurrent Programming in Haskell", O'Reilly Inc., 2013, chapter 2.

Need for speed

Eval monad

```
data Eval a
instance Monad Eval

runEval :: Eval a -> a

rpar :: a -> Eval a
rseq :: a -> Eval a
```

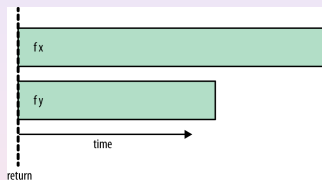
Remarks

- ▶ `rpar`: argument could be evaluated in parallel
- ▶ `rseq`: evaluate argument and wait for the result
- ▶ `runEval`: perform computation and return the result

Combining `rpar` and `rseq`

`rpar/rpar`

```
runEval $ do
  a <- rpar (f x)
  b <- rpar (f y)
  return (a,b)
```

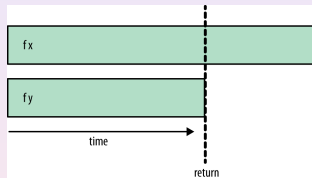


`f x` and `f y` begin to evaluate in parallel;
`return` proceeds immediately
the rest goes on while `f x` and `f y` are evaluated

Combining rpar and rseq

rpar/rseq

```
runEval $ do
  a <- rpar (f x)
  b <- rseq (f y)
  return (a,b)
```

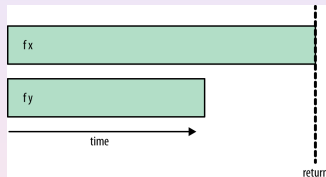


`f x` and `f y` begin to evaluate in parallel;
`return` waits for `f y`

Combining rpar and rseq

rpar/rseq/rseq

```
runEval $ do
  a <- rpar (f x)
  b <- rseq (f y)
  rseq a
  return (a,b)
```



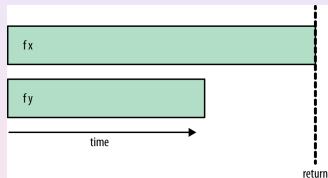
f x and f y begin to evaluate in parallel;
return waits for both f y and f x

Combining rpar and rseq

rpar/rseq/rseq/rseq

```
runEval $ do
  a <- rpar (f x)
  b <- rpar (f y)
  rseq a
  rseq b
  return (a,b)
```

f x and f y begin to evaluate in parallel;
return waits for both f y and f x
"Most symmetrical" version



Example

```
import Control.Parallel
import Control.Parallel.Strategies

fib 0 = 0
fib 1 = 1
fib n = fib (n-1) + fib (n-2)

test1 = do
  x <- rpar (fib 36)
  y <- rpar (fib 35)
  return (x,y)
```

Example (Test results)

	return	last computation completed
rpar/rpar	0.00s	1.35s
rpar/rseq	0.79s	1.31s
rpar/rseq/rseq	1.37s	1.37s
rpar/rpar/rseq/rseq	1.37s	1.37s

Summary

- ▶ Monads allow doing effectful programming in a controlled manner